

라벨링 결과 분석

라벨링 결과 및 Rule 베이스 객체탐지 가능성 확인

1. 라벨링 환경 세팅

- label-studio 설치
- label-studio 시스템 변수 설정
 1. 윈도우 검색 : “시스템 환경 변수 편집”
 2. “환경변수” 버튼 선택
 3. 시스템 변수 하단의 “새로 만들기” 버튼 선택
 4. 변수 이름: “LABEL_STUDIO_LOCAL_FILES_DOCUMENT_ROOT”, 변수 값: “import 할 폴더 경로” 입력 후 확인 버튼 선택
- 프로젝트 생성
 1. create 버튼 선택
 2. 프로젝트 이름 및 기본 정보 입력
 3. Settings의 Labeling Interface에서 Browse Templates 선택
 4. Computer Vision 카테고리의 Object Detection with Bounding Boxes로 선택
 5. Add label names의 Add를 선택해 라벨링 카테고리 설정
 - “id”: 0, “name”: “chemical”
 - “id”: 1, “name”: “table”
 - “id”: 2, “name”: “title”
 6. 상단 과정 후 Save 버튼 선택

2. Data import

- Source Cloud Storage 설정
 1. Settings의 Cloud Storage 선택
 2. Add Source Storage
 3. Local files type으로 Absolute local path, title 작성
 4. Treat every bucket object as a source file 체크 후 Add Storage
 5. Sync Storage로 동기화 진행

3. 라벨링 진행

- 약 1500장 데이터 라벨링 진행
 - 3개의 프로젝트로 나눠서 라벨링 진행

4. Data export

- YOLO format으로 export
 - ex) 2 0.493638910616032 0.0820476858345021 0.6719795150501673 0.035063113604488085 와 같이 저장

4-1. YOLO(You Only Look Once) 모델 설명

- 특징

1. 단일 스테이지 방식
 - 이미지를 한 번만 보고 객체의 위치와 클래스를 동시에 예측
2. 빠른 처리 속도
 - 단일 신경망으로 직접 bounding box와 클래스 확률을 예측
- 장점
 1. 높은 처리 속도와 실시간 성능
 2. 전체적인 맥락을 고려한 예측으로 높은 정확도
 3. 간단한 파이프라인으로 쉬운 구현과 학습
 4. 다양한 환경에서의 일반화 성능이 우수
- 형식
 - [class_id, x_center, y_center, width, height] 형식의 문자열 리스트로 라벨링 값 저장
 - class_id를 제외한 xywh는 0~1 사이의 정규화된 값

5. 라벨링 결과 확인

```
[ ]: # 필수 라이브러리 불러오기
```

```
import os
import json
import colorsys
from PIL import Image, ImageDraw, ImageFont
```

```
[ ]: def generate_distinct_color(n):
```

```
    """
```

```
        n개의 구별되는 색상을 생성하는 함수
```

```
    Args:
```

```
        n (int): 생성할 색상의 수
```

```
    Returns:
```

```
        list: RGB 튜플의 리스트, 각 튜플은 (r, g, b) 형식의 0~1 사이의 범위의 값
```

```
    """
```

```
    hue_partition = 1.0 / (n+1)
```

```
    # 색 상환을 균등하게 나누는 계산
```

```
    return [colorsys.hsv_to_rgb(hue_partition * i, 1.0, 1.0) for i in range(n)]
```

```
[ ]: def read_categories(json_path):
```

```
    """
```

```
        JSON 파일에서 카테고리 정보를 읽어오는 함수
```

```
    Args:
```

```
        json_path (str): 카테고리 정보가 담긴 JSON 파일의 경로
```

```
    Returns:
```

```
        tuple: 두 개의 딕셔너리를 포함하는 튜플
```

```

- category_dict: {카테고리 ID : 카테고리 이름}
- color_dict: {카테고리 ID : (R, G, B)} (R, G, B는 0~255 범위의 정
수)
"""
with open(json_path, 'r', encoding='utf-8') as file:
    data = json.load(file)
    categoris = data.get('categories', [])

category_dict = {item['id']: item['name'] for item in categoris}
colors = generate_distinct_color(len(category_dict))
color_dict = {id: tuple(int(x * 255) for x in color) for (id, color) in
zip(category_dict.keys(), colors)}
return category_dict, color_dict

```

```

[ ]: def read_yolo_labels(laembl_path):
    """
    YOLO 형식의 레이블 파일을 읽는 함수

    Args:
        laembl_path (str): YOLO 형식 레이블 파일의 경로

    Returns:
        list: 각 객체의 레이블 정보를 담은 리스트
        : [class_id, x_center, y_center, width, height] 형식의 문자열 리스트
    """
    with open(label_path, 'r') as file:
        lines = file.readlines()
    return [line.strip().split() for line in lines]

```

5-1. 라벨링 결과 변환 과정

- 중심점 계산
 - x_center 과 y_center 은 0~1 사이의 정규화된 값
 - x_center 과 y_center 각각에 이미지 크기를 곱해 실제 픽셀 위치로 변환
- 너비와 높이 계산
 - width와 height은 0~1 사이의 정규화된 값
 - width와 height 각각에 이미지 크기를 곱해 실제 픽셀 위치로 변환
- 좌상단과 우상단 좌표 계산
 - 좌상단(left, top) $left = x_center - width/2$
 $top = y_center - height/2$
 - 우하단(right, bottom) $right = x_center + width/2$
 $bottom = y_center + height/2$

```

[ ]: def draw_boxes(image_path, label_path, output_path, categories, color_dict):
    """
    이미지에 바운딩 박스와 레이블을 그리는 함수

```

```

    Args:
        image_path (str): 원본 이미지 파일의 경로
        label_path (str): YOLO 형식 레이블 파일의 경로
        output_path (str): 처리된 이미지를 저장할 경로
        categories (dict): {카테고리 ID : 카테고리 이름}
        color_dict (dict): {카테고리 ID : (R, G, B)} (R, G, B는 0~255 범위의
정수)
    """

    image = Image.open(image_path)
    draw = ImageDraw.Draw(image)
    try:
        font = ImageFont.truetype("arial.ttf", 90)
    except IOError:
        font = ImageFont.load_default()

    img_width, img_height = image.size

    labels = read_yolo_labels(label_path)

    for label in labels:
        class_id, x_center, y_center, width, height = map(float, label)
        class_id = int(class_id)

        left = int((x_center - width/2) * img_width)
        top = int((y_center - height/2) * img_height)
        right = int((x_center + width/2) * img_width)
        bottom = int((y_center + height/2) * img_height)

        class_name = categories.get(class_id, f"Unknown-{class_id}")
        color = color_dict.get(class_id, (128,128,128))
        draw.rectangle([left, top, right, bottom], outline=color, width = 10)

        text_bbox = draw.textbbox((left, top - 25), class_name, font = font)
        draw.rectangle(text_bbox, fill = color)
        draw.text((left, top - 25), class_name, fill = "White", font = font)
    image.save(output_path)

```

[]: # 폴더 경로 및 카테고리 파일 경로 설정 후 영역 표시 함수 실행 코드

```

image_folder = "./image"
label_folder = "./labels"
output_folder = "./result"
json_path = "notes.json"

categories, color_dict = read_categories(json_path)

```

```

for filename in os.listdir(image_folder):
    if filename.endswith(".png"):
        image_path = os.path.join(image_folder, filename)
        label_path = os.path.join(label_folder, os.path.splitext(filename)[0] +
↪ ".txt")
        output_path = os.path.join(output_folder, filename)

        if os.path.exists(label_path):
            draw_boxes(image_path, label_path, output_path, categories,
↪ color_dict)
            print(f"Processed: {filename}")
        else:
            print(f"file not found for : : {filename}")

```

6. Rule 베이스 객체탐지 가능성 확인

6-1. Rule 베이스 객체탐지 가능성 확인 방법

1. 라벨링 이미지 데이터를 양식별 1~2장씩 수집
 - 총 80장으로, 약 40장의 이미지 데이터 양식이 있음을 알 수 있음
2. 가로, 세로 대표 이미지 임의로 한 장씩 선정
3. 이미지 파일의 사이즈를 통해 가로, 세로 이미지 데이터 구분
4. 약 80개 이미지 파일의 라벨링 결과(.txt)를 가로, 세로 구분하여 가로, 세로 대표 이미지에 바운딩 박스를 누적해서 표시

[]: # 필수 라이브러리 불러오기

```

import os
import json
from PIL import Image, ImageDraw
import colorsys

```

[]: #아래 함수는 기존 코드 사용

```

def generate_distinct_color(n):
def read_categories(json_path):
def read_yolo_labels(label_path):

```

[]: `def draw_boxes(draw, labels, img_width, img_height, color_dict):`

```

    """
        기존 코드에서 categories 텍스트 출력 부분을 제거하고 코드를 재활용
    """
    for label in labels:
        class_id, x_center, y_center, width, height = map(float, label)
        class_id = int(class_id)

        left = int((x_center - width/2) * img_width)

```

```

top = int((y_center - height/2) * img_height)
right = int((x_center + width/2) * img_width)
bottom = int((y_center + height/2) * img_height)

color = color_dict.get(class_id, (128,128,128))
draw.rectangle([left, top, right, bottom], outline=color, width = 5)

```

```

[ ]: def draw_category_legend(draw, categories, color_dict, img_width):
    """
        결과 이미지 우측 상단에 색상별 카테고리를 표시하기 위한 함수
    """

    try:
        font = ImageFont.truetype("arial.ttf", 50)
    except IOError:
        font = ImageFont.load_default()

    start_x = img_width - 300
    start_y = 20

    for category_id, category_name in categories.items():
        color = color_dict.get(int(category_id), (128,128,128))

        draw.rectangle([start_x, start_y, start_x + 40, start_y + 40],
            ↪fill=color, outline = (0,0,0))

        draw.text((start_x + 50, start_y + 5), category_name, fill=(0,0,0),
            ↪font = font)

        start_y+= 45

```

```

[ ]: def process_images(image_folder, label_folder, landscape_image_path,
    ↪portrait_image_path, output_folder, categories, color_dict):
    """
        가로, 세로 대표 파일에 라벨링 결과를 누적해서 그리는 함수
    """

    landscape_image = Image.open(landscape_image_path)
    portrait_image = Image.open(portrait_image_path)

    landscape_draw = ImageDraw.Draw(landscape_image)
    portrait_draw = ImageDraw.Draw(portrait_image)

    draw_category_legend(landscape_draw, categories, color_dict,
    ↪landscape_image.width)
    draw_category_legend(portrait_draw, categories, color_dict, portrait_image.
    ↪width)

```

```

for filename in os.listdir(image_folder):
    if filename.endswith(".png"):
        image_path = os.path.join(image_folder, filename)
        label_path = os.path.join(label_folder, os.path.splitext(filename)
[0] + ".txt")

        if os.path.exists(label_path):
            with Image.open(image_path) as img:
                img_width, img_height = img.size
                labels = read_yolo_labels(label_path)

                if img_width > img_height:
                    draw_boxes(landscape_draw, labels, img_width, img_height,
↪color_dict)
                else:
                    draw_boxes(portrait_draw, labels, img_width, img_height,
↪color_dict)

                print(f"Processed: {filename}")
            else:
                print(f"file not found for : : {filename}")
        landscape_image.save(os.path.join(output_folder, "combined_landscape_result.
↪png"))
        portrait_image.save(os.path.join(output_folder, "combined_portrait_result.
↪png"))

```

```

[ ]: image_folder = "./image"
    label_folder = "./labels"
    output_folder = "./result"
    json_path = "notes.json"
    landscape_image_path = "가로 대표 이미지 경로"
    portrait_image_path = "세로 대표 이미지 경로"

    categories, color_dict = read_categories(json_path)

    # 코드 실행
    process_images(image_folder, label_folder, landscape_image_path,
↪portrait_image_path, output_folder, categories, color_dict)

```

6-2. Rule 베이스 객체탐지 결과

```

[1]: import matplotlib.pyplot as plt
    from PIL import Image

    img1 = Image.open('i_types_result\combined_portrait_result.png')

```